



INFORME DEL TRABAJO

I. PORTADA

Tema:	Listas
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Tisalema Guashco Darwin Joel
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DEL TRABAJO

2.1 Objetivos

General:

Diseñar e implementar en Java diversas variantes y aplicaciones de listas circulares enlazadas, comprendiendo su estructura interna, sus ventajas frente a las listas lineales y su utilidad para resolver problemas reales que requieren un recorrido cíclico continuo.

Específicos:

- Implementar las operaciones fundamentales de una lista circular enlazada (inserción al inicio/final/posición específica, eliminación por posición/valor, recorrido y conteo de elementos), garantizando que la estructura mantenga siempre su propiedad circular.
- Aplicar la lista circular a la resolución de problemas clásicos de la ciencia de la computación, simulación de turnos bajo el algoritmo Round-Robin y el problema de Josephus, analizando por qué esta estructura de datos resulta la más adecuada para modelarlos.
- Desarrollar un caso de uso aplicado a software real (un sistema de reproducción musical circular), demostrando la correspondencia directa entre una estructura de datos vista de forma teórica en clase y su implementación en un producto de software funcional.

2.2 Ejercicio

Ejercicio 1: Lista Simple Enlazada Circular

1. Ejercicio

Implemente en Java una lista simplemente enlazada circular que permita: insertar al inicio, insertar al final, mostrar todos los elementos, verificar si la lista está vacía y contar el número de elementos. Como requisito adicional, explique con un ejemplo por qué en una lista circular el último nodo debe apuntar al primero.

2. Solución propuesta

Se crea una clase ListaCircular que administra internamente objetos Nodo. El programa contiene dos clases:

Clase	Responsabilidad
-------	-----------------



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

Nodo	Clase interna que guarda un dato entero y la referencia al siguiente nodo.
ListaCircular	TDA que administra el círculo de nodos y expone las operaciones públicas.

3. ¿Qué representa esta estructura?

Esta estructura define: los datos que almacena (enteros dentro de nodos), las operaciones permitidas (insertar al inicio/final, mostrar, contar, verificar vacío) y la regla que siempre debe cumplirse (el último nodo siempre enlaza de vuelta al primero). El estado interno es:

```
public class ListaCircular {  
    private Nodo cabeza; // primer nodo  
    private Nodo cola;   // ultimo nodo  
    private int contador; // cantidad de elementos  
}
```

Los datos están encapsulados porque cabeza, cola y contador son privados. Para trabajar con ellos se utilizan métodos públicos: lista.insertarAlFinal(dato); lista.mostrar(); lista.estaVacia();

4. Relación entre requisitos y código

Requisito	Aplicación en el código
Estructura circular	cola.siguiente siempre se reasigna hacia cabeza en cada inserción.
Datos primitivos	int utilizado para el dato de cada nodo y para el contador.
Clase	Nodo (interna) y ListaCircular.
Encapsulamiento	Atributos private, acceso solo mediante métodos públicos.
Recorrido sin null	El método mostrar() usa un ciclo do-while que compara contra cabeza.
Comentarios	El código incluye comentarios que marcan cada decisión de diseño.

5. Paso 1: crear la clase interna Nodo

```
private class Nodo {  
    int dato;  
    Nodo siguiente;  
    Nodo(int dato) { this.dato = dato; }  
}
```



Nodo es una clase interna porque solo tiene sentido dentro de ListaCircular; ningún código externo necesita crear nodos directamente.

6. Paso 2: declarar el estado de ListaCircular

```
private Nodo cabeza;  
private Nodo cola;  
private int contador;
```

cabeza y cola permiten insertar en $O(1)$ tanto al inicio como al final, sin recorrer la lista completa.

7. Paso 3: insertar manteniendo la propiedad circular

```
public void insertarAlInicio(int dato) {  
    Nodo nuevo = new Nodo(dato);  
    if (estaVacia()) {  
        cabeza = nuevo; cola = nuevo;  
        nuevo.siguiente = nuevo;    // unico nodo: se apunta a si mismo  
    } else {  
        nuevo.siguiente = cabeza;  
        cabeza = nuevo;  
        cola.siguiente = cabeza;    // re-enlazar el circulo  
    }  
    contador++;  
}
```

La línea clave es `cola.siguiente = cabeza`: sin ella, la lista dejaría de ser circular apenas se insertara un segundo elemento.

8. Paso 4: recorrer la lista sin usar null

```
Nodo actual = cabeza;  
do {  
    // procesar actual.dato  
    actual = actual.siguiente;  
} while (actual != cabeza);
```

Como en esta lista nunca existe un null entre los nodos, el recorrido debe detenerse comparando contra cabeza, no contra null.

9. Paso 5: verificar vacío y contar elementos

```
public boolean estaVacia() { return cabeza == null; }  
public int contar() { return contador; }
```

10. Paso 6: insertar al final

```
public void insertarAlFinal(int dato) {  
    Nodo nuevo = new Nodo(dato);  
    if (estaVacia()) {  
        cabeza = nuevo; cola = nuevo; nuevo.siguiente = nuevo;  
    } else {  
        cola.siguiente = nuevo;
```



```
nuevo.siguiete = cabeza; // cierra el circulo hacia la cabeza
cola = nuevo;
}
contador++;
}
```

11. Paso 7: instanciar y probar el programa

```
ListaCircular lista = new ListaCircular();
lista.insertarAlFinal(2);
lista.insertarAlFinal(3);
lista.insertarAlInicio(1);
lista.insertarAlFinal(4);
lista.mostrar();
```

12. Tipos de datos utilizados

Tipo	Categoría	Variables
int	Primitivo	dato, contador
Nodo	Referencia	cabeza, cola, siguiete

13. Diagrama de clases

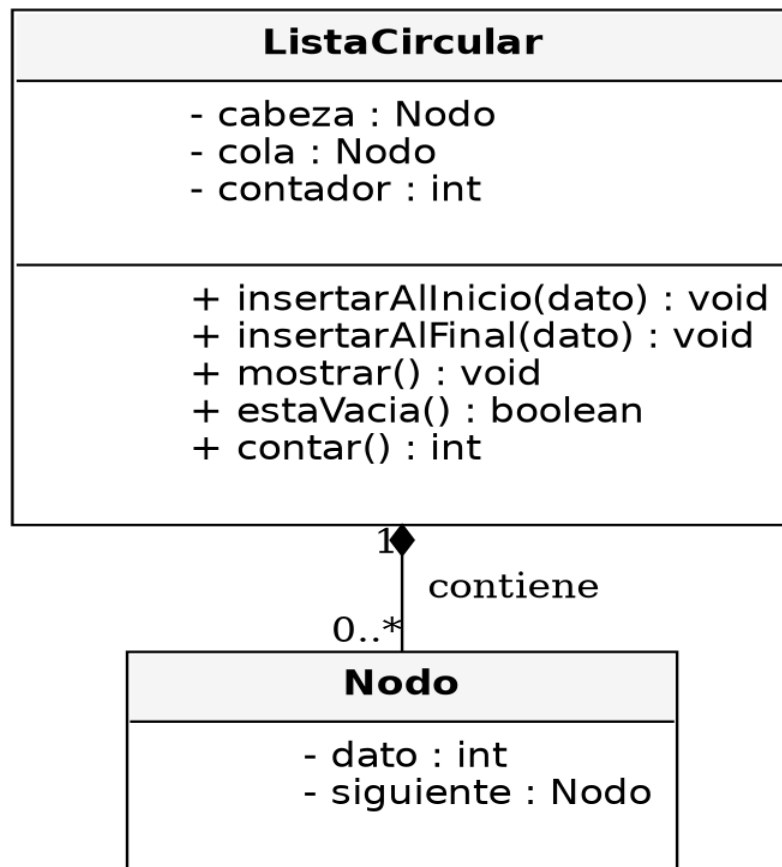


Figura 1. Diagrama de clases de ListaCircular y Nodo (ejercicio 1).

14. Especificación del TDA

Estado

- cabeza: referencia al primer nodo.
- cola: referencia al último nodo; su siguiente siempre apunta a cabeza.
- contador: cantidad de nodos actuales.

Operaciones

Método	Resultado
insertarAlInicio(dato)	Agrega un nodo como nueva cabeza.
insertarAlFinal(dato)	Agrega un nodo como nueva cola.
mostrar()	Imprime todos los elementos en orden.
estaVacia() / contar()	Consultan el estado sin modificarlo.



Invariantes

- El último nodo (cola) siempre enlaza con el primero (cabeza).
- Si la lista tiene un único nodo, este se enlaza consigo mismo.
- contador siempre coincide con la cantidad real de nodos presentes.

15. Pseudocódigo

FUNCION insertarAlFinal(dato)

 crear nuevo nodo

 SI lista vacia ENTONCES cabeza <- nuevo ; cola <- nuevo ; nuevo.siguiete <- nuevo

 SINO cola.siguiete <- nuevo ; nuevo.siguiete <- cabeza ; cola <- nuevo

 contador <- contador + 1

FIN FUNCION

16. Prueba de escritorio

Operación	Resultado
estaVacia()	true (lista recién creada)
insertarAlFinal(2); insertarAlFinal(3); insertarAlInicio(1); insertarAlFinal(4);	1 -> 2 -> 3 -> 4 -> (vuelve a 1)
contar()	4

¿Por qué el último nodo apunta al primero?

En una lista lineal, el último nodo apunta a null. En una circular, apunta de vuelta al primero. Ejemplo con 1 -> 2 -> 3 -> 4: si fuera lineal, el nodo 4 apuntaría a null y recorrerla en bucle exigiría una condición extra para volver al inicio. Al ser circular, 4.siguiete apunta directamente a 1, por lo que el recorrido continúa solo:

1 -> 2 -> 3 -> 4 -> (vuelve a 1) -> 2 -> 3 -> 4 -> ...



Ejercicio 2: Inserción y Eliminación Controlada

1. Ejercicio

Desarrolle un programa en Java que implemente una lista circular y permita: insertar un elemento en una posición específica, eliminar un elemento por posición, eliminar un elemento por valor, e imprimir la lista antes y después de cada operación. Explique qué ocurre en los casos: lista vacía, lista con un solo nodo y lista con varios nodos.

2. Solución propuesta

Se extiende la lógica del ejercicio 1 agregando operaciones indexadas por posición y búsqueda por valor:

Clase	Responsabilidad
Nodo	Clase interna que guarda un dato entero y la referencia al siguiente nodo.
ListaCircular	TDA con operaciones de inserción y eliminación controladas por posición o valor.

3. ¿Qué representa esta estructura?

El TDA amplía sus reglas: además de mantener la circularidad, debe garantizar que cualquier posición entre 0 y contador sea válida, y que eliminar no deje punteros colgantes. El estado interno es el mismo que el ejercicio 1:

```
public class ListaCircular {  
    private Nodo cabeza;  
    private Nodo cola;  
    private int contador;  
}
```

Los datos siguen encapsulados; las nuevas operaciones públicas son insertarEnPosicion(), eliminarPorPosicion() y eliminarPorValor().

4. Relación entre requisitos y código

Requisito	Aplicación en el código
Inserción por posición	insertarEnPosicion(dato, pos) distingue inicio, final y posición intermedia.
Eliminación por posición	eliminarPorPosicion(pos) localiza y desenlaza el nodo.



Eliminación por valor	eliminarPorValor(valor) recorre comparando cada dato.
Impresión antes/después	mostrar() se invoca antes y después de cada operación desde Main.
Caso borde: un nodo	Se maneja aparte en eliminarPorPosicion/Valor (cabeza y cola pasan a null).
Comentarios	El código marca los tres casos borde en cada método.

5. Paso 1: reutilizar la clase Nodo

```
private class Nodo {  
    int dato;  
    Nodo siguiente;  
    Nodo(int dato) { this.dato = dato; }  
}
```

6. Paso 2: declarar insertarEnPosicion()

```
public void insertarEnPosicion(int dato, int posicion) {  
    if (posicion < 0 || posicion > contador) { informar error; return; }  
    // posicion == 0 -> insertar al inicio  
    // posicion == contador -> insertar al final  
    // caso general -> recorrer hasta posicion-1 y enlazar en medio  
}
```

7. Paso 3: mantener el círculo al insertar en medio

```
Nodo anterior = cabeza;  
for (int i = 0; i < posicion - 1; i++) anterior = anterior.siguiente;  
nuevo.siguiente = anterior.siguiente;  
anterior.siguiente = nuevo;
```

A diferencia de insertar al inicio o al final, insertar en medio no requiere tocar cabeza ni cola, solo reenlazar dos punteros locales.

8. Paso 4: eliminar por posición distinguiendo los tres casos

```
if (contador == 1) {  
    cabeza = null; cola = null; // caso: un solo nodo  
} else if (posicion == 0) {  
    cabeza = cabeza.siguiente;  
    cola.siguiente = cabeza; // re-cerrar el círculo  
} else {  
    // recorrer hasta el nodo anterior y desenlazar  
    if (aEliminar == cola) cola = anterior;  
}
```

9. Paso 5: eliminar por valor



```
Nodo actual = cabeza, anterior = cola;  
int vueltas = 0;  
while (vueltas < contador) {  
    if (actual.dato == valor) { /* desenlazar igual que por posicion */ return; }  
    anterior = actual; actual = actual.siguiete; vueltas++;  
}  
informar 'no encontrado';
```

Se usa un contador de vueltas (no comparar contra cabeza) porque, si el valor buscado está en la propia cabeza, comparar actual != cabeza terminaría el ciclo antes de evaluarlo.

10. Paso 6: imprimir antes y después de cada operación

```
System.out.println("Antes:"); lista.mostrar();  
lista.eliminarPorValor(30);  
System.out.println("Despues:"); lista.mostrar();
```

11. Paso 7: instanciar y probar el programa

```
ListaCircular lista = new ListaCircular();  
lista.insertarEnPosicion(20, 0);  
lista.insertarEnPosicion(30, 1);  
lista.insertarEnPosicion(25, 1);  
lista.eliminarPorValor(30);
```

12. Tipos de datos utilizados

Tipo	Categoría	Variables
int	Primitivo	dato, posicion, contador
Nodo	Referencia	cabeza, cola, siguiente

13. Diagrama de clases

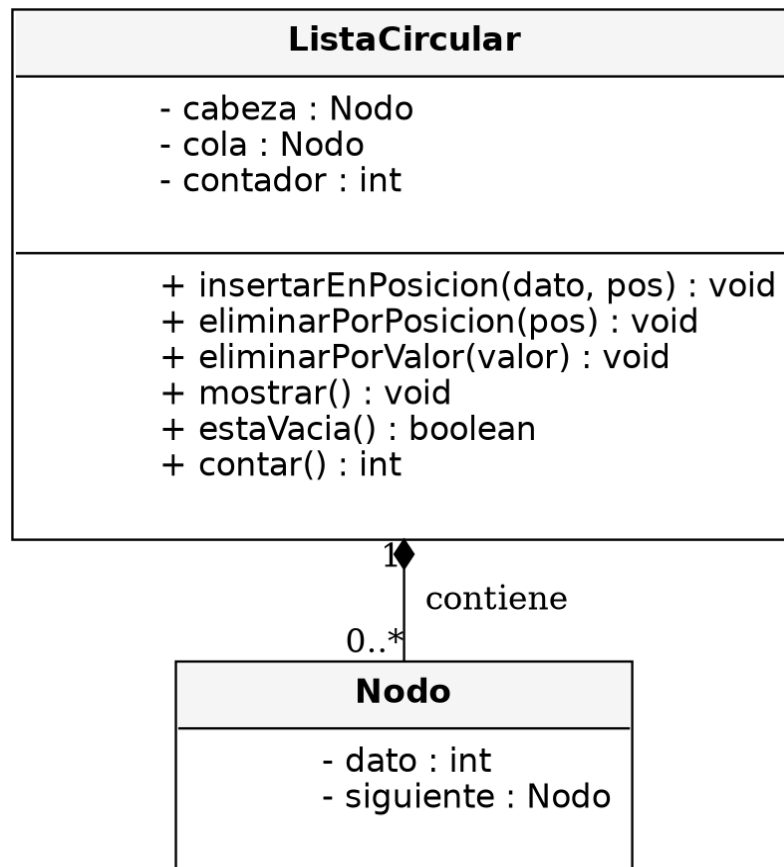


Figura 2. Diagrama de clases de ListaCircular con operaciones por posición y valor (ejercicio 2).

14. Especificación del TDA

Estado

- cabeza, cola: extremos del círculo.
- contador: cantidad de nodos, usado para validar posiciones.

Operaciones

Método	Resultado
insertarEnPosicion(dato, pos)	Inserta en inicio, final o en medio según pos.
eliminarPorPosicion(pos)	Elimina el nodo ubicado en esa posición.
eliminarPorValor(valor)	Elimina la primera coincidencia encontrada.

Invariantes

- $0 \leq \text{posicion} \leq \text{contador}$ para insertar; $0 \leq \text{posicion} < \text{contador}$ para eliminar.
- Tras cualquier eliminación, si contador llega a 0, cabeza y cola quedan en null.



- El círculo permanece cerrado después de cada operación.

15. Pseudocódigo

```
FUNCION eliminarPorValor(valor)
SI lista vacia ENTONCES informar y salir
SI contador = 1 ENTONCES
    SI cabeza.dato = valor ENTONCES vaciar lista
    SINO informar 'no encontrado'
SINO
    recorrer comparando cada nodo con valor
    SI se encuentra ENTONCES desenlazar y ajustar cabeza/cola
    SINO informar 'no encontrado'
FIN FUNCION
```

16. Prueba de escritorio

Operación	Antes	Después
insertarEnPosicion(10, 0) [vacía]	(vacía)	10 -> (vuelve a 10)
insertarEnPosicion(30, 1) [final]	20 -> (vuelve a 20)	20 -> 30 -> (vuelve a 20)
eliminarPorValor(30)	20 -> 25 -> 30 -> (vuelve a 20)	20 -> 25 -> (vuelve a 20)
eliminarPorPosicion(0)	20 -> 25 -> (vuelve a 20)	25 -> (vuelve a 25)

Explique qué ocurre en los siguientes casos: lista vacía / un nodo / varios nodos

Lista vacía: no hay nada que eliminar; se detecta con esta `Vacia()` y se informa sin tocar punteros. Un solo nodo: al eliminarlo, cabeza y cola deben quedar en null a la vez, porque no queda ningún nodo válido al cual apuntar. Varios nodos: el caso general reenlaza el nodo anterior con el siguiente del eliminado, cuidando los bordes de operar sobre la posición 0 o sobre la cola.



Ejercicio 3: Simulación de Turnos Round-Robin

1. Ejercicio

Construya un programa en Java que simule la atención de procesos usando una lista circular bajo el algoritmo Round-Robin. Cada proceso tiene nombre y tiempo restante; el quantum es de 2 unidades; si el proceso no termina, vuelve al final del ciclo; si termina, se elimina de la lista. Muestre paso a paso el orden de ejecución y el estado de la lista después de cada turno.

2. Solución propuesta

Se modela cada proceso como un nodo con nombre y tiempoRestante. El programa contiene dos clases:

Clase	Responsabilidad
Nodo	Clase interna con nombre, tiempoRestante y la referencia al siguiente proceso.
ListaProcesos	TDA que administra el círculo de procesos y ejecuta la simulación Round-Robin.

3. ¿Qué representa esta estructura?

Esta estructura representa una cola circular de procesos: los datos son el nombre y el tiempo restante de CPU de cada proceso; la operación central reparte quantum de tiempo de forma rotativa. El estado interno es:

```
public class ListaProcesos {  
    private Nodo cabeza;  
    private Nodo cola;  
}
```

Los procesos se agregan con agregar (nombre, tiempo) y la simulación completa se dispara con ejecutarRoundRobin(quantum), sin exponer los nodos internos.

4. Relación entre requisitos y código

Requisito	Aplicación en el código
Quantum de 2 unidades	Math.min(quantum, actual.tiempoRestante) limita el tiempo consumido por turno.



Proceso no termina -> vuelve al final	Simplemente se avanza actual al siguiente nodo (la circularidad ya lo manda al final de la cola).
Proceso termina -> se elimina	Se desenlaza el nodo reconectando el anterior con el siguiente.
Datos primitivos	int para tiempoRestante.
Clase / instanciación	Nodo y ListaProcesos; se instancian procesos con agregar().
Comentarios	El código marca el punto donde el proceso termina o continúa.

5. Paso 1: crear la clase interna Nodo con datos del proceso

```
private class Nodo {  
    String nombre;  
    int tiempoRestante;  
    Nodo siguiente;  
    Nodo(String nombre, int tiempo) { this.nombre = nombre; this.tiempoRestante = tiempo; }  
}
```

6. Paso 2: agregar procesos al círculo

```
public void agregar(String nombre, int tiempo) {  
    Nodo nuevo = new Nodo(nombre, tiempo);  
    if (estaVacia()) { cabeza = nuevo; cola = nuevo; nuevo.siguiente = nuevo; }  
    else { cola.siguiente = nuevo; nuevo.siguiente = cabeza; cola = nuevo; }  
}
```

7. Paso 3: aplicar el quantum en cada turno

```
int ejecutado = Math.min(quantum, actual.tiempoRestante);  
actual.tiempoRestante -= ejecutado;
```

Si al proceso le quedan menos de 2 unidades, se ejecuta solo lo que le falta (no más); si le quedan 2 o más, se ejecutan exactamente 2.

8. Paso 4: decidir si el proceso continúa o termina

```
Nodo siguienteNodo = actual.siguiente;  
if (actual.tiempoRestante <= 0) {  
    // termino: se saca del círculo  
    anterior.siguiente = actual.siguiente;  
    if (actual == cabeza) cabeza = actual.siguiente;  
    if (actual == cola) cola = anterior;  
    actual = siguienteNodo;  
} else {  
    // continua: avanza al final del ciclo (round-robin)  
    anterior = actual; actual = actual.siguiente;  
}
```



9. Paso 5: repetir hasta vaciar la lista

```
while (!estaVacia()) {  
    // ejecutar un turno (pasos 3 y 4)  
    mostrar();  
}
```

10. Paso 6: ejecutar la simulación completa

```
public void ejecutarRoundRobin(int quantum) {  
    // bucle principal descrito en el paso 5  
}
```

11. Paso 7: instanciar procesos y probar el programa

```
ListaProcesos lista = new ListaProcesos();  
lista.agregar("P1", 5);  
lista.agregar("P2", 3);  
lista.agregar("P3", 4);  
lista.ejecutarRoundRobin(2);
```

12. Tipos de datos utilizados

Tipo	Categoría	Variables
int	Primitivo	tiempoRestante, quantum
String	Referencia	nombre
Nodo	Referencia	cabeza, cola, siguiente

13. Diagrama de clases

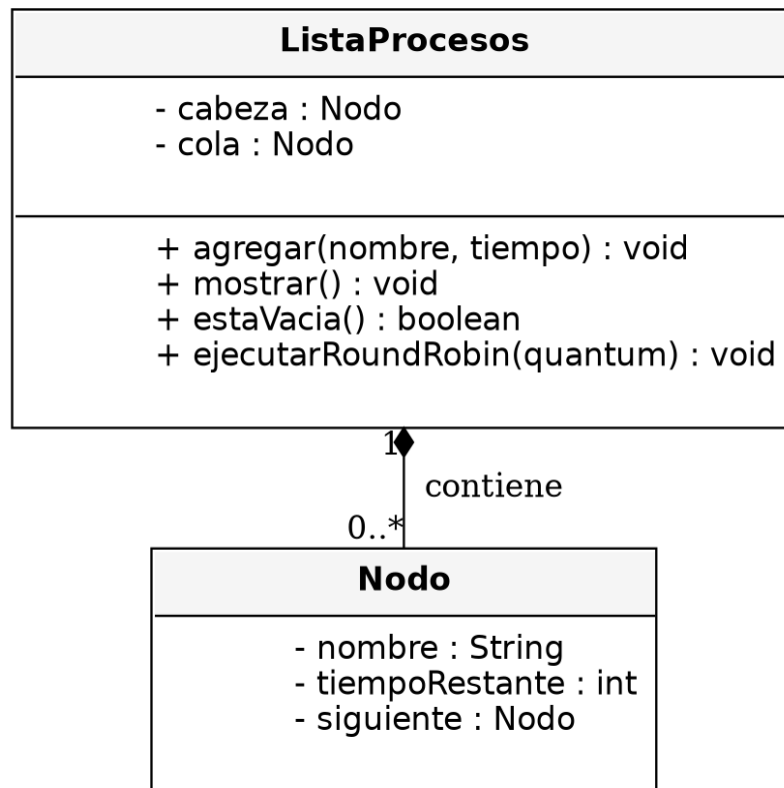


Figura 3. Diagrama de clases de ListaProcesos y Nodo (ejercicio 3).

14. Especificación del TDA

Estado

- cabeza, cola: extremos del círculo de procesos.
- cada nodo guarda su propio tiempoRestante, que disminuye turno a turno.

Operaciones

Método	Resultado
agregar(nombre, tiempo)	Inserta un nuevo proceso al final del círculo.
ejecutarRoundRobin(quantum)	Simula la asignación cíclica de CPU hasta vaciar la lista.

Invariantes

- tiempoRestante nunca es negativo (se limita con Math.min).
- Un proceso solo sale de la lista cuando su tiempoRestante llega a 0.
- El círculo permanece cerrado mientras haya más de un proceso activo.

15. Pseudocódigo

```
FUNCION ejecutarRoundRobin(quantum)
    actual <- cabeza
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

MIENTRAS lista no vacia HACER
 ejecutado <- min(quantum, actual.tiempoRestante)
 actual.tiempoRestante <- actual.tiempoRestante - ejecutado
 SI actual.tiempoRestante <= 0 ENTONCES eliminar actual del circulo
 SINO avanzar actual al siguiente nodo
FIN MIENTRAS
FIN FUNCION

16. Prueba de escritorio

Turno	Proceso ejecutado	Estado tras el turno
1	P1 (usa 2, restante 3)	P1(3) -> P2(3) -> P3(4)
2	P2 (usa 2, restante 1)	P1(3) -> P2(1) -> P3(4)
3	P3 (usa 2, restante 2)	P1(3) -> P2(1) -> P3(2)
4	P1 (usa 2, restante 1)	P1(1) -> P2(1) -> P3(2)
5	P2 (usa 1, restante 0) TERMINA	P1(1) -> P3(2)
6	P3 (usa 2, restante 0) TERMINA	P1(1)
7	P1 (usa 1, restante 0) TERMINA	(vacía)



Ejercicio 4: Problema de Josephus

1. Ejercicio

Implemente en Java el problema de Josephus usando una lista circular: dadas n personas ubicadas en círculo, elimine cada k -ésima persona hasta que quede solo una. Pruebe con $n=5$, $k=2$ y con $n=7$, $k=3$; muestre el orden de eliminación e indique el sobreviviente final.

2. Solución propuesta

Se arma un círculo de n nodos numerados de 1 a n , y se recorre eliminando cada k -ésimo nodo. El programa contiene dos clases:

Clase	Responsabilidad
Nodo	Clase interna que guarda el número de la persona y la referencia al siguiente nodo.
Josephus	Clase que arma el círculo y ejecuta el algoritmo de eliminación.

3. ¿Qué representa esta estructura?

Esta estructura representa literalmente el enunciado: n personas ubicadas en círculo. El algoritmo no requiere guardar estado adicional entre llamadas, por lo que `resolver(n, k)` construye la lista y la resuelve en la misma operación:

```
public class Josephus {  
    public void resolver(int n, int k) {  
        // arma el círculo y elimina cada k-esimo  
    }  
}
```

No se exponen los nodos: `resolver(n, k)` imprime directamente el orden de eliminación y el sobreviviente.

4. Relación entre requisitos y código

Requisito	Aplicación en el código
n personas en círculo	Se crean n nodos numerados 1.. n y se cierra el círculo (último $\rightarrow$ primero).
Eliminar cada k -ésima	Un ciclo for avanza $k-1$ pasos antes de cada eliminación.
Hasta que quede una sola	El ciclo principal se repite mientras <code>actual.siguiente != actual</code> .



Mostrar orden de eliminación	Se imprime cada persona justo antes de desenlazarla.
Datos primitivos	int para el número de cada persona y para n y k.
Comentarios	El código marca dónde se cuenta, dónde se elimina y dónde se detecta el sobreviviente.

5. Paso 1: crear la clase interna Nodo

```
private class Nodo {  
    int persona;  
    Nodo siguiente;  
    Nodo(int persona) { this.persona = persona; }  
}
```

6. Paso 2: armar el círculo de n personas

```
Nodo cabeza = new Nodo(1);  
Nodo actual = cabeza;  
for (int i = 2; i <= n; i++) { actual.siguiente = new Nodo(i); actual = actual.siguiente; }  
actual.siguiente = cabeza; // cierra el círculo
```

7. Paso 3: avanzar k-1 pasos contando personas

```
for (int i = 1; i < k; i++) {  
    previo = actual;  
    actual = actual.siguiente;  
}
```

Se necesitan dos referencias (previo y actual) porque, al ser una lista simplemente enlazada, eliminar un nodo requiere conocer al que lo precede.

8. Paso 4: eliminar a la k-ésima persona

```
System.out.print(actual.persona + " ");  
previo.siguiente = actual.siguiente; // saca a "actual" del círculo  
actual = previo.siguiente; // el conteo sigue desde aquí
```

9. Paso 5: repetir hasta que quede una sola persona

```
while (actual.siguiente != actual) {  
    // pasos 3 y 4  
}
```

La condición `actual.siguiente != actual` detecta el momento en que un único nodo quedó apuntándose a sí mismo: ese es el sobreviviente.

10. Paso 6: mostrar el sobreviviente final

```
System.out.println("Sobreviviente: " + actual.persona);
```

11. Paso 7: instanciar y probar con ambos casos



```
Josephus j = new Josephus();  
j.resolver(5, 2); // n=5, k=2  
j.resolver(7, 3); // n=7, k=3
```

12. Tipos de datos utilizados

Tipo	Categoría	Variables
int	Primitivo	persona, n, k
Nodo	Referencia	cabeza, actual, previo, siguiente

13. Diagrama de clases

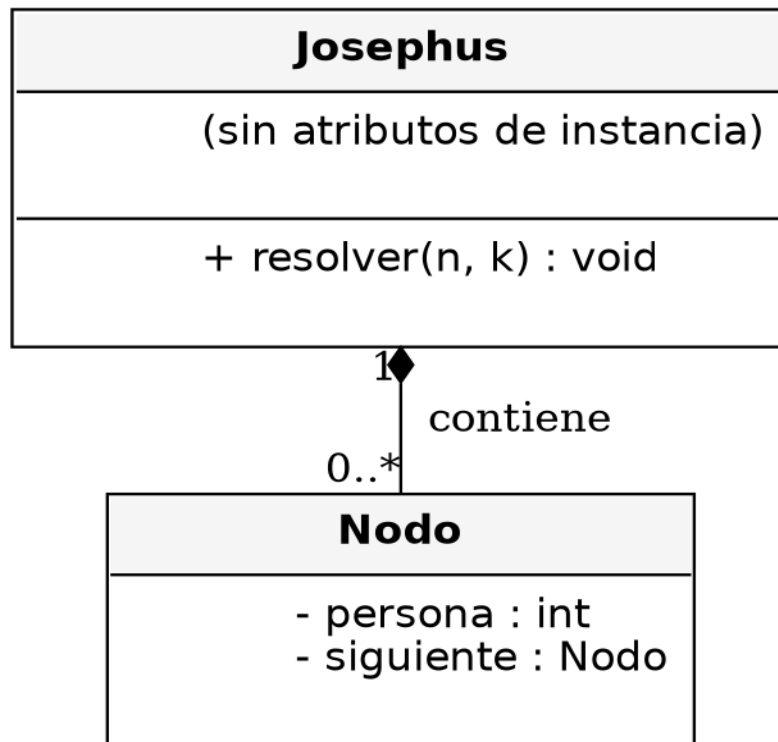


Figura 4. Diagrama de clases de Josephus y Nodo (ejercicio 4).

14. Especificación del TDA

Estado

- El círculo se construye y se consume dentro de una sola llamada a resolver(n, k).

Operaciones



Método	Resultado
resolver(n, k)	Arma el círculo de n personas y elimina cada k-ésima hasta dejar una sola.

Invariantes

- El círculo siempre queda cerrado tras armar las n personas.
- Cada eliminación reduce el círculo en un nodo, sin romper la circularidad.
- El proceso termina cuando queda exactamente un nodo (se apunta a sí mismo).

15. Pseudocódigo

```
FUNCION resolver(n, k)
  armar circulo de n nodos (1..n)
  actual <- cabeza ; previo <- cola
  MIENTRAS actual.siguiente != actual HACER
    repetir (k-1) veces: previo <- actual ; actual <- actual.siguiente
    mostrar actual como eliminado
    previo.siguiente <- actual.siguiente
    actual <- previo.siguiente
  FIN MIENTRAS
  mostrar actual como sobreviviente
FIN FUNCION
```

16. Prueba de escritorio

Caso	Orden de eliminación	Sobreviviente
n = 5, k = 2	2, 4, 1, 5	3
n = 7, k = 3	3, 6, 2, 7, 5, 1	4

¿Por qué una lista circular es adecuada aquí?

El problema describe personas en círculo, con un conteo que da vueltas indefinidamente hasta que queda una sola. La lista circular modela esto sin condiciones especiales: el conteo avanza con `actual = actual.siguiente` sin verificar null, y al eliminar, el conteo se reanuda directamente desde `previo.siguiente`, que ya es el nodo correcto gracias al re-enlace del círculo.



Ejercicio 5: Caso Aplicado - Sistema de Reproducción Circular

1. Ejercicio

Diseñe un programa en Java que represente una playlist musical circular. El sistema debe permitir agregar canciones al inicio y al final, mostrar la playlist completa, reproducir la siguiente canción, eliminar una canción por nombre, y volver automáticamente a la primera canción al llegar al final.

2. Solución propuesta

Se modela cada canción como un nodo con su nombre. El programa contiene dos clases:

Clase	Responsabilidad
Nodo	Clase interna que guarda el nombre de la canción y la referencia a la siguiente.
Playlist	TDA que administra el círculo de canciones y el puntero de reproducción actual.

3. ¿Qué representa esta estructura?

A diferencia de los ejercicios anteriores, esta estructura agrega una tercera referencia, actual, que indica qué canción se está reproduciendo. El estado interno es:

```
public class Playlist {  
    private Nodo cabeza;  
    private Nodo cola;  
    private Nodo actual; // cancion que se esta reproduciendo  
}
```

El avance de la reproducción se controla únicamente a través de reproducirSiguiente(), sin exponer actual directamente.

4. Relación entre requisitos y código

Requisito	Aplicación en el código
Agregar al inicio/final	agregarAlInicio() / agregarAlFinal(), igual que en el ejercicio 1.
Mostrar playlist completa	mostrarPlaylist() marca entre corchetes la canción actual.



Reproducir siguiente	reproducirSiguiente() hace actual = actual.siguiente.
Volver sola al inicio	No requiere código especial: el círculo ya conecta la última canción con la primera.
Eliminar por nombre	eliminarPorNombre() reasigna actual si la canción eliminada era la que sonaba.
Comentarios	El código marca el punto donde el círculo produce el retorno automático.

5. Paso 1: crear la clase interna Nodo

```
private class Nodo {  
    String cancion;  
    Nodo siguiente;  
    Nodo(String cancion) { this.cancion = cancion; }  
}
```

6. Paso 2: agregar canciones al inicio y al final

```
public void agregarAlInicio(String cancion) { /* igual patron que ejercicio 1 */ }  
public void agregarAlFinal(String cancion) {  
    Nodo nuevo = new Nodo(cancion);  
    if (estaVacia()) { cabeza = nuevo; cola = nuevo; nuevo.siguiente = nuevo; actual = nuevo; }  
    else { cola.siguiente = nuevo; nuevo.siguiente = cabeza; cola = nuevo; }  
}
```

7. Paso 3: mostrar la playlist marcando la canción actual

```
sb.append(n == actual ? "[" + n.cancion + "]" : n.cancion);
```

8. Paso 4: avanzar la reproducción

```
public void reproducirSiguiente() {  
    if (estaVacia()) { informar; return; }  
    actual = actual.siguiente; // el circulo hace que vuelva a cabeza automaticamente  
    System.out.println("Reproduciendo: " + actual.cancion);  
}
```

9. Paso 5: eliminar una canción por nombre

```
if (n.cancion.equals(cancion)) {  
    prev.siguiente = n.siguiente;  
    if (n == cabeza) cabeza = n.siguiente;  
    if (n == cola) cola = prev;  
    if (n == actual) actual = n.siguiente; // si sonaba esta, pasa a la siguiente  
}
```

10. Paso 6: manejar el caso de un solo nodo al eliminar



```
if (cabeza == cola) {  
    if (cabeza.cancion.equals(cancion)) { cabeza = null; cola = null; actual = null; }  
    else informar 'no encontrada';  
}
```

11. Paso 7: instanciar y probar el programa

```
Playlist playlist = new Playlist();  
playlist.agregarAlFinal("Cancion A");  
playlist.agregarAlFinal("Cancion B");  
playlist.agregarAlFinal("Cancion C");  
playlist.agregarAlInicio("Cancion Inicial");  
playlist.reproducirSiguiente();
```

12. Tipos de datos utilizados

Tipo	Categoría	Variables
String	Referencia	cancion
Nodo	Referencia	cabeza, cola, actual, siguiente

13. Diagrama de clases

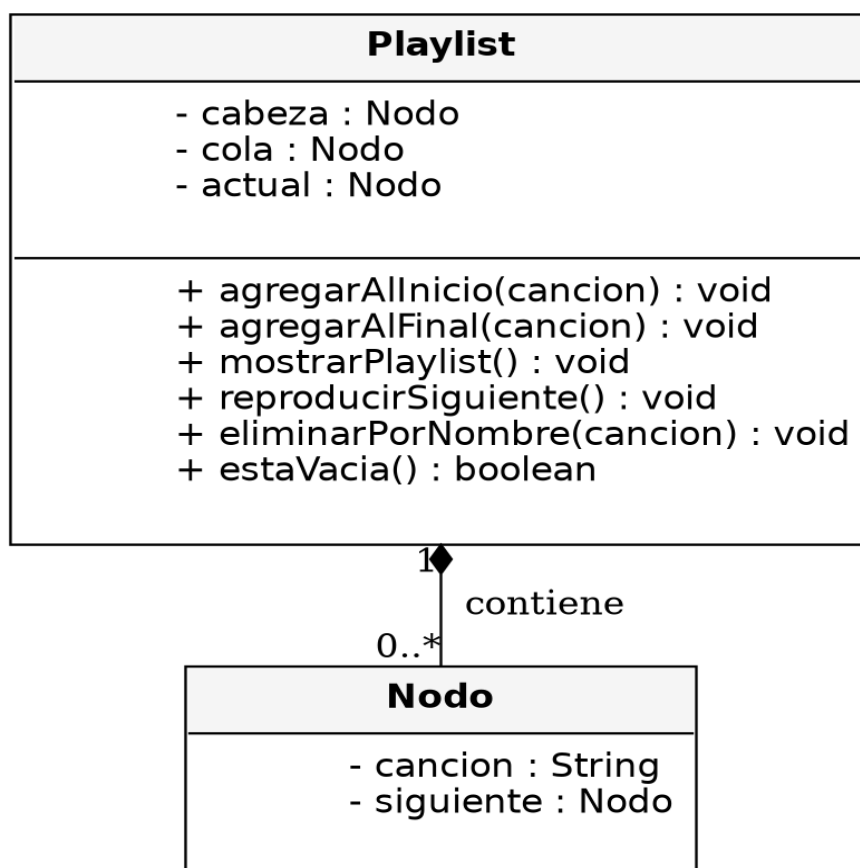




Figura 5. Diagrama de clases de Playlist y Nodo (ejercicio 5).

14. Especificación del TDA

Estado

- cabeza, cola: extremos del círculo de canciones.
- actual: referencia a la canción que está sonando en este momento.

Operaciones

Método	Resultado
agregarAlInicio / agregarAlFinal	Insertan una canción manteniendo el círculo cerrado.
mostrarPlaylist()	Imprime todas las canciones, marcando la actual.
reproducirSiguiente()	Avanza actual al siguiente nodo.
eliminarPorNombre(cancion)	Busca y desenlaza la canción indicada.

Invariantes

- actual siempre apunta a un nodo válido mientras la playlist no esté vacía.
- Al eliminar la canción que suena, actual se reasigna antes de perder la referencia.
- El círculo permanece cerrado tras cada inserción o eliminación.

15. Pseudocódigo

```
FUNCION reproducirSiguiente()  
SI lista vacia ENTONCES informar y salir  
actual <- actual.siguiente  
mostrar 'Reproduciendo: ' + actual.cancion  
FIN FUNCION
```

16. Prueba de escritorio

Acción	Resultado
Agregar (final): A, B, C — Agregar (inicio): Inicial	Inicial -> [A] -> B -> C
reproducirSiguiente() x4	B -> C -> Inicial -> A (vuelve sola al llegar al final)
eliminarPorNombre("B") [canción que sonaba]	Inicial -> A -> [C] (actual se reasigna sola)



Ventaja de la lista circular frente a una lista lineal

En una lista lineal, el último nodo apunta a null, por lo que al llegar a la última canción el recorrido simplemente termina. Para lograr que la reproducción "vuelva a empezar" automáticamente, habría que agregar una condición extra en el código: verificar si el puntero llegó a null y, en ese caso, reasignarlo manualmente de vuelta al primer nodo (cabeza). Esto añade una excepción especial al flujo normal de recorrido.

En una lista **circular**, esa condición no es necesaria: el último nodo (cola) ya está enlazado directamente con el primero (cabeza), de modo que `reproducirSiguiente()` simplemente avanza el puntero con `enReproduccion = enReproduccion.siguiente`, y al llegar al final, este mismo avance ya lo devuelve de forma natural a la primera canción. El comportamiento de "repetir la playlist" queda incorporado en la propia estructura de datos, no en lógica adicional del programa.

Además, en cuanto a inserción y eliminación, ambas estructuras insertan y eliminan en tiempo $O(1)$ al mantener referencias a cabeza y cola, pero en la lista circular esas operaciones deben además preservar el enlace de cierre (`cola.siguiente = cabeza`) en cada cambio; a cambio de ese pequeño costo, se obtiene una estructura que modela fielmente el comportamiento cíclico que pide el enunciado (recorrido indefinido en bucle), algo que una lista lineal no puede representar sin lógica externa.

La ventaja principal es que la lista circular traslada la regla de negocio "al llegar al final, vuelve al inicio" a la propia estructura de datos, evitando condicionales adicionales y reduciendo la posibilidad de errores al gestionar manualmente el reinicio del recorrido, tal como exige el enunciado de este ejercicio.

2.3 Conclusiones

- La lista circular evita condiciones especiales para "volver al inicio", ya que el último nodo enlaza directamente con el primero.
- Los casos de borde (lista vacía y de un solo nodo) son los más propensos a errores, pues cabeza y cola pueden coincidir o desaparecer a la vez.
- Problemas como Round-Robin y Josephus se resuelven de forma natural con esta estructura, porque representan literalmente un grupo dispuesto en círculo.
- El caso de la playlist confirma que esta estructura tiene aplicación directa en software real, no solo académico.

2.4 Referencias Bibliográficas

[1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," Baeldung, may 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>. [Accedido: 26-ago-2026].



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

[2] "The Cost of Garbage Collection for State Machine Replication," arXiv preprint, arXiv:2405.11182, 2024. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>. [Accedido: 26-ago-2026].

2.5 Anexos

```
--- LISTA CIRCULAR ---
1. Insertar al inicio
2. Insertar al final
3. Mostrar elementos
4. Verificar si esta vacia
5. Contar elementos
0. Salir
Opcion: 2
Dato a insertar al final: 3

--- LISTA CIRCULAR ---
1. Insertar al inicio
2. Insertar al final
3. Mostrar elementos
4. Verificar si esta vacia
5. Contar elementos
0. Salir
Opcion: 3
2 -> 1 -> 3 -> (vuelve a 2)

--- LISTA CIRCULAR ---
1. Insertar al inicio
2. Insertar al final
3. Mostrar elementos
4. Verificar si esta vacia
5. Contar elementos
0. Salir
Opcion: 4
¿Esta vacia? false

--- LISTA CIRCULAR ---
1. Insertar al inicio
2. Insertar al final
3. Mostrar elementos
4. Verificar si esta vacia
5. Contar elementos
0. Salir
Opcion: 5
Elementos: 3
```

Figura 6. Ejecución del primer ejercicio



```
Dato: 2
Posicion (0..1): 1
Antes de insertar:
Lista: 1 -> (vuelve a 1)
Despues de insertar:
Lista: 1 -> 2 -> (vuelve a 1)

--- LISTA CIRCULAR ---
1. Insertar en posicion
2. Eliminar por posicion
3. Eliminar por valor
0. Salir
Opcion: 1
Dato: 3
Posicion (0..2): 2
Antes de insertar:
Lista: 1 -> 2 -> (vuelve a 1)
Despues de insertar:
Lista: 1 -> 2 -> 3 -> (vuelve a 1)

--- LISTA CIRCULAR ---
1. Insertar en posicion
2. Eliminar por posicion
3. Eliminar por valor
0. Salir
Opcion: 2
Posicion a eliminar: 1
Antes de eliminar:
Lista: 1 -> 2 -> 3 -> (vuelve a 1)
Despues de eliminar:
Lista: 1 -> 3 -> (vuelve a 1)

--- LISTA CIRCULAR ---
1. Insertar en posicion
2. Eliminar por posicion
3. Eliminar por valor
0. Salir
Opcion: 3
Valor a eliminar: 3
Antes de eliminar:
Lista: 1 -> 3 -> (vuelve a 1)
Despues de eliminar:
Lista: 1 -> (vuelve a 1)
```

Figura 7. Ejecución del segundo ejercicio



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
Cuantos procesos desea registrar: 3
Nombre del proceso 1: A
Tiempo requerido de A: 5
Nombre del proceso 2: B
Tiempo requerido de B: 2
Nombre del proceso 3: C
Tiempo requerido de C: 9

Estado inicial:
  Lista: A(5) -> B(2) -> C(9)

--- SIMULACION ROUND-ROBIN (quantum = 2) ---
Turno 1: ejecuta A (usa 2 u., restante 3)
  Lista: A(3) -> B(2) -> C(9)
Turno 2: ejecuta B (usa 2 u., restante 0)
  B termino y sale de la lista.
  Lista: A(3) -> C(9)
Turno 3: ejecuta C (usa 2 u., restante 7)
  Lista: A(3) -> C(7)
Turno 4: ejecuta A (usa 2 u., restante 1)
  Lista: A(1) -> C(7)
Turno 5: ejecuta C (usa 2 u., restante 5)
  Lista: A(1) -> C(5)
Turno 6: ejecuta A (usa 1 u., restante 0)
  A termino y sale de la lista.
  Lista: C(5)
Turno 7: ejecuta C (usa 2 u., restante 3)
  Lista: C(3)
Turno 8: ejecuta C (usa 2 u., restante 1)
  Lista: C(1)
Turno 9: ejecuta C (usa 1 u., restante 0)
  C termino y sale de la lista.
  Lista: (vacía, todos los procesos terminaron)
Todos los procesos han finalizado.
```

Figura 8. Ejecución del primer ejercicio



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
Numero de personas (n): 5
Cada cuantas personas se elimina (k): 2
n = 5, k = 2
Orden de eliminacion: 2 4 1 5
Sobreviviente: 3
PS C:\Users\ASUS TUF\Documents\C++\LISTAS> ^C
PS C:\Users\ASUS TUF\Documents\C++\LISTAS>
PS C:\Users\ASUS TUF\Documents\C++\LISTAS> c++; cd 'c:\
e-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-
be5cde\redhat.java\jdt_ws\LISTAS_592c81ee\bin' 'Ejercic
Numero de personas (n): 7
Cada cuantas personas se elimina (k): 3
n = 7, k = 3
Orden de eliminacion: 3 6 2 7 5 1
Sobreviviente: 4
PS C:\Users\ASUS TUF\Documents\C++\LISTAS> 
```

Figura 9. Ejecución del primer ejercicio



```
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 3
Playlist: B -> [A] -> C
```

```
--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 4
Reproduciendo: C
```

```
--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 4
Reproduciendo: B
```

```
--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 4
Reproduciendo: A
```

Figura 10. Ejecución del primer ejercicio – 1ra parte



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

```
--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 4
Reproduciendo: A

--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 4
Reproduciendo: C

--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 5
Nombre de la cancion a eliminar: B
Cancion eliminada: B

--- PLAYLIST CIRCULAR ---
1. Agregar cancion al inicio
2. Agregar cancion al final
3. Mostrar playlist
4. Reproducir siguiente
5. Eliminar cancion por nombre
0. Salir
Opcion: 3
Playlist: A -> [C]
```

Figura 11. Ejecución del primer ejercicio – da parte



UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026

A continuación, se debe adjuntar el enlace del repositorio donde se podrá visualizar los códigos en Java correspondientes a los 5 ejercicios desarrollados (Lista Circular Básica, Inserción y Eliminación Controlada, Simulación Round-Robin, Problema de Josephus y Playlist Circular), además de las capturas de ejecución de cada programa y el presente documento en formato .docx y .pdf. Los diagramas de clases (UML) de cada ejercicio ya se encuentran incorporados dentro del documento, en la sección correspondiente a cada uno.

<https://github.com/Joel03032007/ESTRUCTURA--TAREA-03-Listas>